

前言

webpack优化，高级知识后续才会学，目前只学基础

一.node.js

1.概念（下定义）

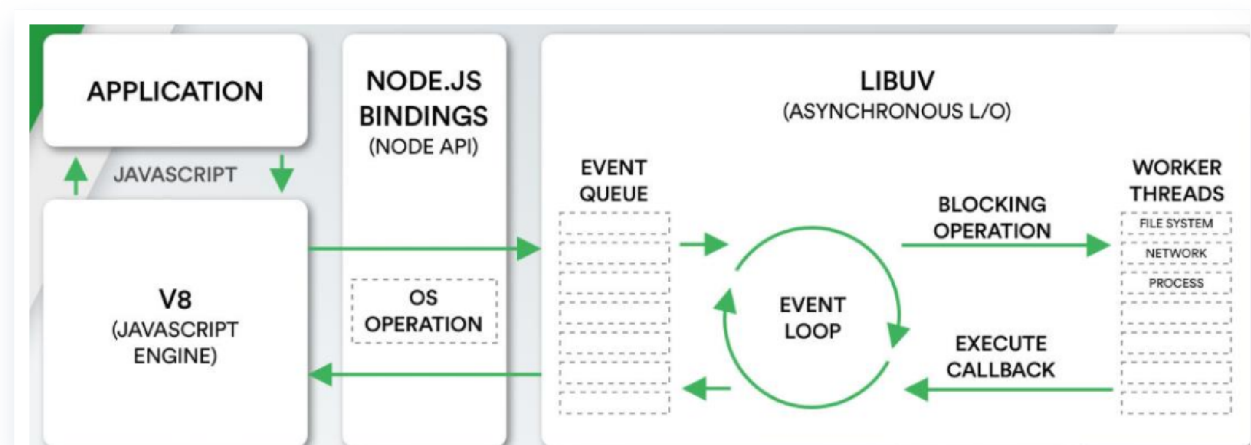
□ Node.js是一个基于V8 JavaScript引擎的JavaScript运行时环境。

■ 也就是说Node.js基于V8引擎来执行JavaScript的代码，但是不仅仅只有V8引擎：

- 前面我们知道V8可以嵌入到任何C++应用程序中，无论是Chrome还是Node.js，事实上都是嵌入了V8引擎来执行JavaScript代码；
- 但是在Chrome浏览器中，还需要解析、渲染HTML、CSS等相关渲染引擎，另外还需要提供支持浏览器操作的API、浏览器自己的事件循环等；
- 另外，在Node.js中我们也需要进行一些额外的操作，比如文件系统读/写、网络IO、加密、压缩解压文件等操作；

■ 我们来看一个单独的Node.js的架构图：

- 我们编写的JavaScript代码会经过V8引擎，再通过Node.js的Bindings，将任务放到Libuv的事件循环中；
- libuv（Unicorn Velociraptor—独角伶盗龙）是使用C语言编写的库；
- libuv提供了事件循环、文件系统读写、网络IO、线程池等等内容；



2.应用

- 应用一：目前前端开发的库都是以node包的形式进行管理；
- 应用二：npm、yarn、pnpm工具成为前端开发使用最多的工具；
- 应用三：越来越多的公司使用Node.js作为web服务器开发、中间件、代理服务器；
- 应用四：大量项目需要借助Node.js完成前后端渲染的同构应用；
- 应用五：资深前端工程师需要为项目编写脚本工具（前端工程师编写脚本通常会使用JavaScript，而不是Python或者shell）；
- 应用六：很多企业在使用Electron来开发桌面应用程序；

❑ **LTS版本**：（Long-term support, 长期支持）相对稳定一些，推荐线上环境使用该版本；

❑ **Current版本**：最新的Node版本，包含很多新特性；

[Download](#)

3.基本使用



Node的输出

■ console.log

- 最常用的输入内容的方式: `console.log`

■ console.clear

- 清空控制台: `console.clear`

■ console.trace

- 打印函数的调用栈: `console.trace`

全局对象:

浏览器的全局对象是window，并且用var定义的变量会成为window的属性

node的全局对象是global，但是用var定义的变量不会成为全局对象的属性

- 这些全局对象，我们并不需要从一开始全部一个个学习;
- 某些全局对象并不常用;
- 某些全局对象我们会在后续学习中讲到;
 - ✓ 比如module、exports、require()会在模块化中讲到;
 - ✓ 比如Buffer后续会专门讲到;

目前掌握的可以有: `__pathename` `__filename`

全局对象还有一些方法, `setTimeout` `setInterval` `setImmed`
`iate` `process.nextTick`

还需要了解什么是 `process`

Node.js 中的 process 是一个全局对象，它提供有关当前 Node.js 进程的信息，并对其进行控制。process 对象具有许多方法和属性，可以用于处理进程相关的任务，例如：

- 获取命令行参数
- 设置环境变量
- 退出进程
- 监听进程事件
- 设置进程标题
- 发送信号给进程

通过 process 对象，可以访问诸如进程参数、环境变量、标准输入输出流等信息，并且可以与操作系统进行交互，执行一些系统级的操作。

二.模块化

■ 到底什么是模块化、模块化开发呢？

- 事实上模块化开发最终的目的是将程序划分成一个个小的结构；
- 这个结构中编写属于自己的逻辑代码，有自己的作用域，定义变量名词时不会影响到其他的结构；
- 这个结构可以将自己希望暴露的变量、函数、对象等导出给其他结构使用；
- 也可以通过某种方式，导入另外结构中的变量、函数、对象等；

■ 上面说提到的结构，就是模块；按照这种结构划分开发程序的过程，就是模块化开发的过程；

□ 但是JavaScript本身，直到ES6（2015）才推出了自己的模块化方案；

□ 在此之前，为了让JavaScript支持模块化，涌现出了很多不同的模块化规范：AMD、CMD、CommonJS等；

在es6之前都是社区提出来的

1.commonJS

- 我们需要知道CommonJS是一个规范，最初提出是在浏览器以外的地方使用，并且当时被命名为ServerJS，后来为了体现它的广泛性，修改为CommonJS，平时我们也会简称为CJS。
 - Node是CommonJS在服务器端一个具有代表性的实现；
 - Browserify是CommonJS在浏览器中的一种实现；
 - webpack打包工具具备对CommonJS的支持和转换；
- 所以，Node中对CommonJS进行了支持和实现，让我们在开发node的过程中可以方便的进行模块化开发：
 - 在Node中每一个js文件都是一个单独的模块；
 - 这个模块中包括CommonJS规范的核心变量：exports、module.exports、require；
 - 我们可以使用这些变量来方便的进行模块化开发；
- 前面我们提到过模块化的核心是导出和导入，Node中对其进行了实现：
 - exports和module.exports可以负责对模块中的内容进行导出；
 - require函数可以帮助我们导入其他模块（自定义模块、系统模块、第三方库模块）中的内容；

(1) 例子以及一些本质：

假设有一个名为 `math.js` 的模块，其中包含了一些数学相关的函数。

```
// math.js

// 导出加法函数
exports.add = function(a, b) {
  return a + b;
};

// 导出乘法函数
exports.multiply = function(a, b) {
  return a * b;
};
```

在另一个文件中，我们可以使用 `require` 关键字来引入并使用 `math.js` 模块中的函数。

```
// app.js

// 引入 math.js 模块
var math = require('./math');

// 使用导出的加法函数
var sum = math.add(3, 5);
console.log('Sum:', sum); // 输出: Sum: 8

// 使用导出的乘法函数
var product = math.multiply(4, 7);
console.log('Product:', product); // 输出: Product: 28
```

在这个例子中，`math.js` 模块中定义了两个函数，并通过 `exports` 关键字将它们导出。然后在 `app.js` 中使用 `require` 关键字引入 `math.js` 模块，并通过 `math.add` 和 `math.multiply` 来调用导出的函数。

但是上面如果`math`模块定义的变量很多，导出了很多，那么我们在导入`math`模块之后，要一直`math.xxx`来使用，有没有一种简介的办法？

```
const {sum,multiply,divide,xxx,xxx,xxx} =
require('./math');

multiply(4, 7);

sum(0,1);
```

解构赋值，因为`require`返回的结果本质上是一个`Object`，这个`Object`就是`exports`，里面的键就是导出时候的变量名，即`exports.xxx = xxx`

所以`require`的**本质上**是引用赋值。`require`方法返回的对象共享了这个`exports`的对象内存地址。

因此如果对exports对象的属性值进行改变，或者在require后对于得到的对象属性值的修改，都会改变导入or导出的值。

但是！！！！导入文件or模块内部的变量不会改变！！！！（当然如果是引用类型的变量的话，就会改变，本质原因是引用赋值）！！！！

（2）node中的使用

导出：

在node中，我们一般以这样的方式去导出：

```
module.exports = {  
  name,  
  age,  
  sayHello  
}
```

□ module.exports和exports有什么关系或者区别呢？

■ 我们追根溯源，通过维基百科中对CommonJS规范的解析：

- CommonJS中是没有module.exports的概念的；
- 但是为了实现模块的导出，Node中使用的是Module的类，每一个模块都是Module的一个实例，也就是module；
- 所以在Node中真正用于导出的其实根本不是exports，而是module.exports；
- 因为module才是导出的真正实现者；

■ 但是，为什么exports也可以导出呢？

- 这是因为module对象的exports属性是exports对象的一个引用；
- 也就是说 module.exports = exports = main中的bar；

因为node在实现module类的时候，将module.exports = exports了，即把commonJS的exports对象赋值给了module的exports属性，也就是module.exports存放着exports对象的引用。默认状态下二者指向同样的内存空间。

但是我们由于我们在开发中常见的写法是

```
module.exports = {  
  name,  
  age,  
  sayHello  
}
```

这样又把module.exports获得了一个新对象的引用，不再是原来的exports对象的引用了. 本质上来讲module.exports的内存地址指向新的空间，而不是原来的exports的内存空间了。

导入：

那么，require的查找规则是怎么样的呢？

- 这里我总结比较常见的查找规则：
- 导入格式如下：require(X)

情况一：X是一个Node核心模块，比如path、http

- 直接返回核心模块，并且停止查找

■ 情况二：X是以 ./ 或 ../ 或 / （根目录）开头的

■ 第一步：将X当做一个文件在对应的目录下查找；

□ 1.如果有后缀名，按照后缀名的格式查找对应的文件

□ 2.如果没有后缀名，会按照如下顺序：

- ✓ 1> 直接查找文件X
- ✓ 2> 查找X.js文件
- ✓ 3> 查找X.json文件
- ✓ 4> 查找X.node文件

■ 第二步：没有找到对应的文件，将X作为一个目录

□ 查找目录下面的index文件

- ✓ 1> 查找X/index.js文件
- ✓ 2> 查找X/index.json文件
- ✓ 3> 查找X/index.node文件

■ 如果没有找到，那么报错：not found

■ 情况三：直接是一个X（没有路径），并且X不是一个核心模块

■ /Users/coderwhy/Desktop/Node/TestCode/04_learn_node/05_javascript-module/02_commonjs/main.js中编写 require('why')

```
paths: [
  '/Users/coderwhy/Desktop/Node/TestCode/04_learn_node/05_javascript-module/02_commonjs/node_modules',
  '/Users/coderwhy/Desktop/Node/TestCode/04_learn_node/05_javascript-module/node_modules',
  '/Users/coderwhy/Desktop/Node/TestCode/04_learn_node/node_modules',
  '/Users/coderwhy/Desktop/Node/node_modules',
  '/Users/coderwhy/Desktop/node_modules',
  '/Users/coderwhy/node_modules',
  '/node_modules'
]
```

■ 如果上面的路径中都没有找到，那么报错：not found

模块的加载过程：

■ 结论一：模块在被第一次引入时，模块中的js代码会被运行一次

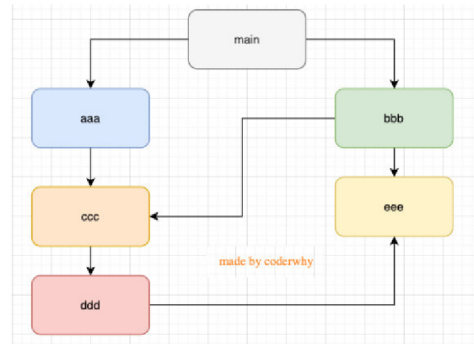
■ 结论二：模块被多次引入时，会缓存，最终只加载（运行）一次

- 为什么只会加载运行一次呢？
- 这是因为每个模块对象module都有一个属性：loaded。
- 为false表示还没有加载，为true表示已经加载；

■ 结论三：如果有循环引入，那么加载顺序是什么？

■ 如果出现右图模块的引用关系，那么加载顺序是什么呢？

- 这个其实是一种数据结构：图结构；
- 图结构在遍历的过程中，有深度优先搜索（DFS, depth first search）和广度优先搜索（BFS, breadth first search）；
- Node采用的是深度优先算法：main -> aaa -> ccc -> ddd -> eee -> bbb



(3) CommonJS规范的缺点

■ CommonJS加载模块是同步的：

- 同步的意味着只有等到对应的模块加载完毕，当前模块中的内容才能被运行；
- 这个在服务器不会有什么问题，因为服务器加载的js文件都是本地文件，加载速度非常快；

■ 如果将它应用于浏览器呢？

- 浏览器加载js文件需要先从服务器将文件下载下来，之后再加载运行；
- 那么采用同步的就意味着后续的js代码都无法正常运行，即使是一些简单的DOM操作；

■ 所以在浏览器中，我们通常不使用CommonJS规范：

- 当然在webpack中使用CommonJS是另外一回事；
- 因为它会将我们的代码转成浏览器可以直接执行的代码；

■ 在早期为了可以在浏览器中使用模块化，通常会采用AMD或CMD：

- 但是目前一方面现代的浏览器已经支持ES Modules，另一方面借助于webpack等工具可以实现对CommonJS或者ES Module代码的转换；
- AMD和CMD已经使用非常少了；

Webpack 选择采用 CommonJS 的模块化规范主要是由于以下几个原因：

1. **Node.js 的普及**：Node.js 是使用 CommonJS 模块系统的流行平台，它推广了 CommonJS 规范。Webpack 作为一个基于 Node.js 构建的工具，采用 CommonJS 可以更好地与 Node.js 生态系统集成。
2. **适合服务端渲染**：Webpack 早期被广泛应用于服务端渲染（SSR）项目中，而 CommonJS 模块化规范正适合在服务器端和客户端通用的场景下使用。

3. 社区支持：Webpack 在早期开发阶段就得到了对 CommonJS 模块化规范的广泛支持，因此选择继续沿用这种规范也是为了避免破坏现有的社区生态。

尽管 CommonJS 在前端领域逐渐被 ES6 模块化规范取代，但由于历史原因和实际需求，Webpack 仍然默认支持 CommonJS 模块化规范，并且可以通过相应的配置支持其他类型的模块化规范。

2.AMD和CMD规范：

■ **AMD主要是应用于浏览器的一种模块化规范：**

- AMD是**Asynchronous Module Definition**（异步模块定义）的缩写；
- 它采用的是**异步加载模块**；
- 事实上AMD的规范还要早于CommonJS，但是CommonJS目前依然在被使用，而AMD使用的较少了；

■ 我们提到过，规范只是定义代码的应该如何去编写，只有有了具体的实现才能被应用：

- AMD实现的比较常用的库是**require.js**和**curl.js**；



CMD规范

■ **CMD规范也是应用于浏览器的一种模块化规范：**

- CMD 是**Common Module Definition**（通用模块定义）的缩写；
- 它也采用的也是**异步加载模块**，但是它将CommonJS的优点吸收了过来；
- 但是目前CMD使用也非常少了；

■ **CMD也有自己比较优秀的实现方案：**

- **SeaJS**

3.ES Module

■ ES Module模块采用export和import关键字来实现模块化：

□ export负责将模块内的内容导出；

□ import负责从其他模块导入内容；

■ 了解：采用ES Module将自动采用严格模式：use strict

(1) 导出：

■ 我们希望将其他中内容全部导出，它可以有如下的方式：

■ 方式一：在语句声明的前面直接加上export关键字

■ 方式二：将所有需要导出的标识符，放到export后面的 {}中

□ 注意：这里的 {}里面不是ES6的对象字面量的增强写法，{}也不是表示一个对象的；

□ 所以： export {name: name}，是错误的写法；

■ 方式三：导出时给标识符起一个别名

□ 通过as关键字起别名

(2) 导入



import关键字

- import关键字负责从另外一个模块中导入内容
- 导入内容的方式也有多种:
 - 方式一: import {标识符列表} from '模块';
 - 注意: 这里的{}也不是一个对象, 里面只是存放导入的标识符列表内容;
 - 方式二: 导入时给标识符起别名
 - 通过as关键字起别名
 - 方式三: 通过 * 将模块功能放到一个模块功能对象 (a module object) 上

注意即使只有一个导入也不可以不用大括号 (除非导出使用了默认导出)

方式三要用对象的方式使用, 比如test.xxx



export和import结合使用

- 补充: export和import可以结合使用

```
export { sum as barSum } from './bar.js';
```

- 为什么要这样做呢?
 - 在开发和封装一个功能库时, 通常我们希望将暴露的所有接口放到一个文件中;
 - 这样方便指定统一的接口规范, 也方便阅读;
 - 这个时候, 我们就可以使用export和import结合使用;

相当于:

```
import {sum} from "./bar.js";
export {sum as barSum};
```

(3) 默认导出

- 前面我们学习的导出功能都是**有名字的导出 (named exports)** :

- 在导出export时指定了名字;
- 在导入import时需要知道具体的名字;

- 还有一种导出叫做**默认导出 (default export)**

- 默认导出export时可以**不需要指定名字**;
- 在**导入时不需要使用 {}**, 并且**可以自己来指定名字**;
- 它也方便我们和现有的CommonJS等规范相互操作;

- **注意: 在一个模块中, 只能有一个默认导出 (default export) ;**

(4) import方法/import函数

codewhy

import函数

- 通过import加载一个模块, 是不可以在其放到逻辑代码中的, 比如:

- 为什么会出现这个情况呢?

- 这是因为ES Module在被JS引擎解析时, 就必须知道它的依赖关系;
- 由于这个时候js代码没有任何的运行, 所以无法在进行类似于if判断中根据代码的执行情况;
- 甚至**拼接路径的写法也是错误的**: 因为我们必须到运行时能确定path的值;

- 但是某些情况下, 我们确实希望动态的来加载某一个模块:

- 如果根据不懂的条件, 动态来选择加载模块的路径;
- 这个时候我们需要使用 **import() 函数**来动态加载;
 - ✓ import函数返回一个Promise, 可以通过then获取结果;

```
if (true) {
  import sub from './modules/foo.js';
}
```

```
let flag = true;
if (flag) {
  import('./modules/aaa.js').then(aaa => {
    aaa.aaa();
  })
} else {
  import('./modules/bbb.js').then(bbb => {
    bbb.bbb();
  })
}
```

这是因为**静态导入**的特性要求在代码解析阶段就能确定导入的模块，以便在编译时进行静态分析和优化。

换句话说，当使用import加载一个模块时，它必须在模块的顶层作用域中，这样可以**确保在代码被解析和执行之前，模块的依赖关系就已经确定**，并且可以正确地进行加载和绑定

4.ESModule原理

ES Module的解析过程可以划分为三个阶段：

- 阶段一：构建（Construction），根据地址查找js文件，并且下载，将其解析成模块记录（Module Record）；
- 阶段二：实例化（Instantiation），对模块记录进行实例化，并且分配内存空间，解析模块的导入和导出语句，把模块指向对应的内存地址。
- 阶段三：运行（Evaluation），运行代码，计算值，并且将值填充到内存地址中；

5. ES6模块与CommonJS模块有什么异同？

- **导入和导出语法**：ES6模块使用import和export语法进行导入和导出，而CommonJS模块使用require()和module.exports语法。
- **静态加载和动态加载**：ES6模块使用静态加载，其中导入和导出语句在编译时解析，使得模块的依赖关系可以在编译时确定。而CommonJS模块使用动态加载，**导入语句在运行时执行**。这是因为CJS导入导出的逻辑是**对象的引用赋值**
- **异步加载**：ES6模块支持异步加载模块（使用import()函数），这使得加载模块更加灵活。CommonJS模块在加载时是同步的，无法异步加载模块。
- **缓存值**：ES6采用实时绑定的方式，导入导出的值都指向同一个内存地址，所以导入值会跟随导出值变化，并且不会缓存值，CommonJS在首次加载模块时后缓存，后续再次加载会执行缓存的结果

怎么实现动态绑定的？

当一个模块被导入时，引用的实际是该模块的命名空间对象，这样就可以实现动态绑定

什么又是命名空间对象？

在 JavaScript 中，通常可以通过对象字面量来创建命名空间对象。

```
// 命名空间对象
const MyNamespace = {
  var1: 'value1',
  var2: 'value2',
  func1: function() {
    // 函数实现
  },
  func2: function() {
    // 函数实现
  }
};

// 访问命名空间对象的属性和方法
console.log(MyNamespace.var1);
MyNamespace.func1();
```

6.css模块化

1. CSS in JS:

- CSS in JS 是一种将 CSS 样式直接写入 JavaScript 文件中的方法。通过使用像 styled-components、Emotion 等库，开发者可以在组件级别内联样式，而不是将样式集中在单独的 CSS 文件中。
- 这种方法的优势之一是可以更好地组织组件和样式，因为它们被紧密耦合在一起。此外，它还允许使用 JavaScript 的强大功能来动态计算样式，以及根据组件状态进行条件渲染样式等。
- 但是，CSS in JS 也有一些争议，因为它可能会增加开发者学习成本，以及使样式与传统的 CSS 工作流程不兼容。

```
import React from 'react';
import styled from 'styled-components';

// 创建一个样式化的组件
const Button = styled.button `background-color: ${props =>
  props.primary ? 'blue' : 'white'};
color: ${props => props.primary ? 'white' : 'blue'};
font-size: 16px;
padding: 10px 20px;
border: 2px solid blue;
border-radius: 5px;`;

// 使用样式化的组件
const MyComponent = () => {
  return (


The image shows two rectangular buttons. The top button is labeled 'Normal Button' and has a white background with a thin grey border. The bottom button is labeled 'Primary Button' and has a blue background with a thin grey border.



```
);
}
```


```

```
export default MyComponent;
```

2. CSS Modules:

- CSS Modules 是一种基于文件名和类名的命名约定，以及构建工具的支持，来实现 CSS 的模块化。
- 使用 CSS Modules，每个 CSS 文件都被视为一个独立的模块，其中的类名会被自动转换成具有唯一性的局部作用域。这意味着在不同模块中使用相同类名不会发生冲突。
- 在构建过程中，构建工具会根据这些约定和规则，自动生成一个映射表，将类名映射到唯一的哈希值，以确保模块间的隔离性。
- CSS Modules 的优点包括简单易用、提供了良好的命名空间隔离和避免了全局作用域的问题，但也需要构建工具的支持。

```
/* styles.module.css */
.button {
  background-color: blue;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
}

/* App.js */
import React from 'react';
import styles from './styles.module.css'; // 导入 CSS 模块

const App = () => {
  return (
```

Click me { /* 使用 CSS 模块的类名 */ }

```
);  
}  
  
export default App;
```

3. Less:

- Less 是一种预处理器，类似于 Sass 和 Stylus，它扩展了 CSS 的功能，提供了诸如变量、嵌套、混合、函数等特性，使得 CSS 更易于维护和扩展。
- Less 可以帮助开发者更加有效地组织和管理样式，提高了代码的重用性和可维护性。
- 尽管 Less 本身并不提供模块化的功能，但结合构建工具如 webpack 或者 gulp 等，可以实现类似 CSS 模块化的效果，通过分离样式文件并进行编译、压缩等处理。

LESS 是一种 CSS 预处理器，它扩展了 CSS 的功能，使得样式表更易于维护和扩展。LESS 允许你使用变量、嵌套规则、混合、运算以及其他功能，然后将其编译成普通的 CSS 文件。以下是使用 LESS 的基本方法：

使用 LESS:

1. **编写 LESS 文件：**编写以 `.less` 扩展名结尾的 LESS 文件，就像编写普通的 CSS 文件一样。你可以在 LESS 文件中使用变量、嵌套规则、混合等功能。
2. **编译 LESS 文件：**
 - 如果你使用 CDN，那么 LESS 文件会在浏览器中动态编译为 CSS 文件。

- 如果你使用构建工具，则可以使用相应的插件或加载器来编译 LESS 文件。

基本语法和功能：

1. **变量**：使用 `@` 符号定义变量，可以在整个样式表中重复使用。

```
@primary-color: #ff0000;
.button {
  color: @primary-color;
}
```

2. **嵌套规则**：可以将选择器嵌套在另一个选择器内，使代码更具可读性。

```
.container {
  h1 {
    color: #333;
  }
}
```

3. **混合**：类似于函数，可以将一组样式作为单个单位进行重用。

```
.rounded-corners {
  border-radius: 5px;
}
.button {
  .rounded-corners();
}
```

4. **运算**：可以在 LESS 中执行简单的数学运算。


```
@base-width: 200px;
.container {
  width: @base-width * 2;
}
```

5. 导入：可以将多个 LESS 文件组合在一起。

```
@import "reset"; // 导入 reset.less 文件
@import "variables"; // 导入 variables.less 文件
```

这只是 LESS 提供的一小部分功能，你可以根据需要深入学习和探索更多的功能和用法。

三.包管理工具

1.npm

包管理工具npm:

□ Node Package Manager, 也就是Node包管理器;

npm管理的包存放在哪里呢？

- 我们发布自己的包其实是发布到registry上面的；
- 当我们安装一个包时其实是从registry上面下载的包；

■ 那么对于一个项目来说，我们如何使用npm来管理这么多包呢？

- 事实上，我们每一个项目都会有一个对应的配置文件，无论是前端项目（Vue、React）还是后端项目（Node）；
- 这个配置文件会记录着你项目的名称、版本号、项目描述等；
- 也会记录着你项目所依赖的其他库的信息和依赖库的版本号；

■ 这个配置文件就是package.json

■ 那么这个配置文件如何得到呢？

- 方式一：手动从零创建项目，npm init -y
- 方式二：通过脚手架创建项目，脚手架会帮助我们生成package.json，并且里面有相关的配置

coderwhy

常见的属性

■ 必须填写的属性：name、version

- name是项目的名称；
- version是当前项目的版本号；
- description是描述信息，很多时候是作为项目的基本描述；
- author是作者相关信息（发布时用到）；
- license是开源协议（发布时用到）；

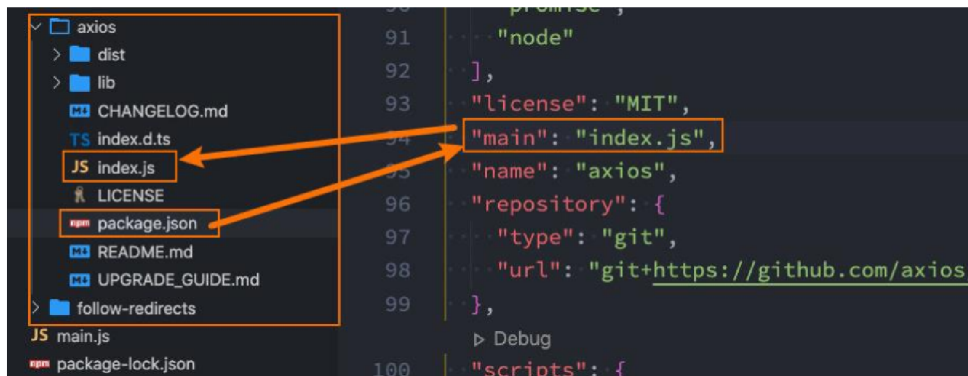
■ private属性：

- private属性记录当前的项目是否是私有的；
- 当值为true时，npm是不能发布它的，这是防止私有项目或模块发布出去的方式；

■ main属性:

□ 设置程序的入口。

- ✓ 比如我们使用axios模块 `const axios = require('axios');`
- ✓ 如果有main属性，实际上是找到对应的main属性查找文件的；



"main" 属性指定了该模块的入口文件。当其他模块引入这个模块时，Node.js 将会使用 "main" 属性指定的文件作为默认的入口点。

■ scripts属性

- scripts属性用于配置一些脚本命令，以键值对的形式存在；
- 配置后我们可以通过 `npm run` 命令的key来执行这个命令；
- npm start和npm run start的区别是什么？
 - ✓ 它们是等价的；
 - ✓ 对于常用的 start、test、stop、restart可以省略掉run直接通过 `npm start` 等方式运行；

也就是说我们使用 `npm run key` 实际上执行的是 value，比如：

"start": "node index.js", 我们执行 `npm run start`, 实际上执行的是 `node index.js`

■ dependencies属性

- dependencies属性是指定无论开发环境还是生成环境都需要依赖的包；
- 通常是我们项目实际开发用到的一些库模块vue、vuex、vue-router、react、react-dom、axios等等；
- 与之对应的是devDependencies；

■ devDependencies属性

- 一些包在生成环境是不需要的，比如webpack、babel等；
- 这个时候我们会通过 `npm install webpack --save-dev`，将它安装到devDependencies属性中；

■ peerDependencies属性

- 还有一种项目依赖关系是对等依赖，也就是你依赖的一个包，它必须是以另外一个宿主包为前提的；
- 比如element-plus是依赖于vue3的，ant design是依赖于react、react-dom；

■ 我们会发现安装的依赖版本出现：^2.0.3或~2.0.3，这是什么意思呢？

■ npm的包通常需要遵从semver版本规范：

- semver: <https://semver.org/lang/zh-CN/>
- npm semver: <https://docs.npmjs.com/misc/semver>

■ semver版本规范是X.Y.Z：

- X主版本号 (major)：当你做了不兼容的 API 修改（可能不兼容之前的版本）；
- Y次版本号 (minor)：当你做了向下兼容的功能性新增（新功能增加，但是兼容之前的版本）；
- Z修订号 (patch)：当你做了向下兼容的问题修正（没有新功能，修复了之前版本的bug）；

■ 我们这里解释一下 ^和~的区别：

- x.y.z：表示一个明确的版本号；
- ^x.y.z：表示x是保持不变的，y和z永远安装最新的版本；
- ~x.y.z：表示x和y保持不变的，z永远安装最新的版本；

■ 局部安装分为开发时依赖和生产时依赖：

```
# 默认安装开发和生产依赖
npm install axios
npm i axios

# 开发依赖
npm install webpack --save-dev
npm install webpack -D
npm i webpack -D

# 根据package.json中的依赖包
npm install
```

■ 全局安装

□ 全局安装是直接将某个包安装到全局：

□ 比如全局安装yarn：

```
npm install yarn -g
```

■ 但是很多人对全局安装有一些误会：

□ 通常使用npm全局安装的包都是一些工具包：yarn、webpack等；

□ 并不是类似于 axios、express、koa等库文件；

□ 所以全局安装了之后并不能让我们在所有的项目中使用 axios等库：

npm 包可以进行全局安装和局部安装，它们在安装位置和使用范围上有所不同。

1. 全局安装：

- 全局安装的包会被安装到系统的全局环境中，而不是特定于某个项目的目录。这意味着，无论当前所在的目录是什么，全局安装的包都可以在命令行中直接调用。
- 全局安装一般适用于那些作为命令行工具使用的包，比如构建工具、脚手架工具等。全局安装的包通常提供了一个命令行命令供开发者在任何地方使用。

2. 局部安装：

- 局部安装的包会被安装到当前项目的 `node_modules` 目录下，它们只对当前项目可见和可用。
- 局部安装通常用于项目特定的依赖，比如项目需要的库、框架等。局部安装的包不会污染全局环境，每个项目都可以有自己独立的依赖。

但是，即使在全局安装了 `axios`，也不能保证在所有项目中都可以使用。这是因为 `axios` 是一个第三方库，它需要在每个项目中单独安装和配置。

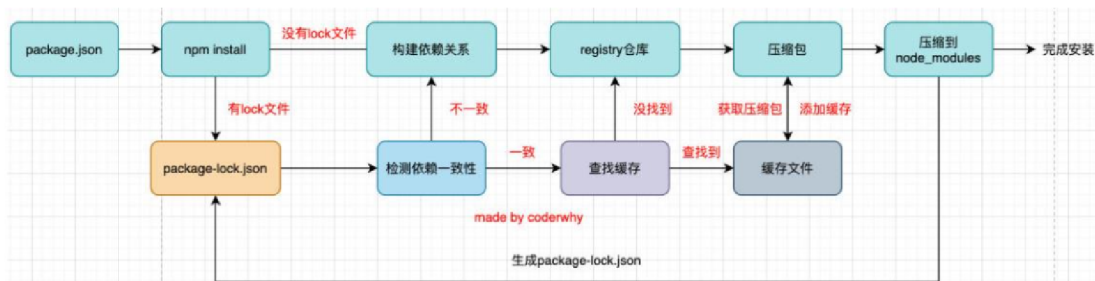


npm install 原理

■ 很多同学之前应该已经会了 `npm install <package>`，但是你是否思考过它的内部原理呢？

- 执行 `npm install` 它背后帮助我们完成了什么操作？
- 我们会发现还有一个称之为 `package-lock.json` 的文件，它的作用是什么？
- 从 npm5 开始，`npm` 支持缓存策略（来自 yarn 的压力），缓存有什么作用呢？

■ 这是一幅我画出的根据 `npm install` 的原理图：



■ npm install会检测是有package-lock.json文件:

□ 没有lock文件

- ✓ 分析依赖关系，这是因为我们可能包会依赖其他的包，并且多个包之间会产生相同依赖的情况；
- ✓ 从registry仓库中下载压缩包（如果我们设置了镜像，那么会从镜像服务器下载压缩包）；
- ✓ 获取到压缩包后会对压缩包进行缓存（从npm5开始有的）；
- ✓ 将压缩包解压到项目的node_modules文件夹中（前面我们讲过，require的查找顺序会在该包下面查找）

□ 有lock文件

- ✓ 检测lock中包的版本是否和package.json中一致（会按照semver版本规范检测）；
 - 不一致，那么会重新构建依赖关系，直接会走顶层的流程；
- ✓ 一致的情况下，会去优先查找缓存
 - 没有找到，会从registry仓库下载，直接走顶层流程；
- ✓ 查找到，会获取缓存中的压缩文件，并且将压缩文件解压到node_modules文件夹中；

2.npx



npx工具

■ npx是npm5.2之后自带的一个命令。

- npx的作用非常多，但是比较常见的是使用它来调用项目中的某个模块的指令。

■ 我们以webpack为例：

- 全局安装的是webpack5.1.3
- 项目安装的是webpack3.6.0

■ 如果我在终端执行 webpack --version使用的是哪一个命令呢？

- 显示结果会是 webpack 5.1.3，事实上使用的是全局的，为什么呢？
- 原因非常简单，在当前目录下找不到webpack时，就会去全局找，并且执行命令；

■ 如何解决这个问题呢？

■ 那么如何使用项目（局部）的webpack，常见的是两种方式：

□ 方式一：明确查找到node_module下面的webpack

□ 方式二：在 scripts定义脚本，来执行webpack；

■ 方式一：在终端中使用如下命令（在项目根目录下）

```
./node_modules/.bin/webpack --version
```

■ 方式二：修改package.json中的scripts

```
"scripts": {  
  "webpack": "webpack --version"  
}
```

■ 方式三：使用npx

```
npx webpack --version
```

■ npx的原理非常简单，它会到当前目录的node_modules/.bin目录下查找对应的命令；

3.pnpm

硬连接（Hard Link）和软连接（Symbolic Link，也称为符号链接）是在文件系统中用来创建文件或目录之间关联的两种方式。

1. 硬连接：

- 硬连接是指多个文件共享同一个索引节点（inode），它们指向相同的存储空间。在创建硬链接时，实际上只是增加了一个文件名指向相同的数据块，删除其中任何一个文件并不会影响其他文件，只有当所有硬链接都被删除时，文件的数据块才会被释放。
- 硬链接不能跨越文件系统，即硬链接只能在同一个文件系统内创建。硬链接可以对普通文件和目录进行创建。

2. 软连接：

- 软连接是创建一个特殊的文件，该文件包含指向另一个文件或目录的路径。软连接实际上创建了一个新的文件，该文件中包含了指向源文件或目录的路径信息。
- 软连接可以跨越文件系统，即软连接可以链接到另一个文件系统中的文件或目录。软连接类似于Windows系统中的快捷方式。

- 当源文件或目录被删除时，软连接仍然存在，但指向的路径将失效，称为“悬空指针”。

总的来说，硬链接与原始文件共享存储空间，而软链接是一个独立的文件，指向另一个文件或目录的路径。**硬链接不能跨越文件系统，而软链接可以。**

原理：



pnpm创建非扁平的 node_modules 目录

■ 当使用 npm 或 Yarn Classic 安装依赖包时，所有软件包都将被提升到 node_modules 的根目录下。

当使用 pnpm 作为包管理工具时，它会采用一种称为“虚拟化”的方式来管理 node_modules 目录，以实现非扁平的依赖结构。在 pnpm 中，每个项目的依赖都被安装在单独的目录中，而不是像 npm 或 Yarn 那样将所有依赖都放在顶层的 node_modules 目录下。

这种虚拟化的方式使得 pnpm 能够实现依赖共享，即多个项目可以共享同一个依赖包的实例，从而减少磁盘占用和加快安装速度。**每个项目的 node_modules 目录中只包含当前项目特定的依赖（通过符号链接指向共享依赖的位置，加快安装速度），而共享的依赖则保存在 pnpm 的全局存储中。（使用硬链接来指向pnpm的全局存储，因为硬链接可以更有效地节省磁盘空间）**

■ 以下是一个与 npm 等价命令的对照表，帮助你快速入门：

npm 命令	pnpm 等价命令
npm install	pnpm install
npm install <pkg>	pnpm add <pkg>
npm uninstall <pkg>	pnpm remove <pkg>
npm run <cmd>	pnpm <cmd>



pnpm的存储store

- 在pnpm7.0之前，统一的存储位置是 ~/.pnpm-store中的；

本地磁盘 (C:) > 用户 > coderwhy > .pnpm-store

- 在pnpm7.0之后，统一的存储位置进行了更改：<pnpm home directory>/store

- 在 Linux 上，默认是 ~/.local/share/pnpm/store
- 在 Windows 上： %LOCALAPPDATA%/pnpm/store
- 在 macOS 上： ~/Library/pnpm/store

本地磁盘 (C:) > 用户 > coderwhy > AppData > Local > pnpm > store >

- 我们可以通过一些终端命令获取这个目录：获取当前活跃的store目录

```
pnpm store path
```

- 另外一个非常重要的store命令是prune（修剪）：从store中删除当前未被引用的包来释放store的空间

```
pnpm store prune
```

四.webpack---打包工具

1.为什么要使用

- 模块化支持：** Webpack 支持 CommonJS、AMD、ES6 等多种模块化规范，能够将各种模块化方式的代码转换成浏览器可识别的代码。
- 资源打包：** Webpack 可以将项目中的各种资源文件，如 JavaScript、CSS、图片、字体等，统一打包管理，减少了在网页中引入各种资源时的手动配置工作。
- 代码拆分：** Webpack 允许将代码拆分成多个 bundle，实现按需加载，提高了页面加载速度和性能。
- 开发环境优化：** Webpack 提供了**开发服务器**、**热模块替换（HMR）**等功能，能够提升开发效率，支持快速的开发调试。
- 生态丰富：** Webpack 拥有庞大的生态系统，有大量的插件和 loader 可以用于处理各种不同类型的文件，比如处理jsx文件转化为浏览器可以识别的html css js文件，比如less文件，将ES6转化为ES5等等

6. 自动化优化： Webpack 提供了丰富的优化功能，如代码压缩、Tree Shaking、代码分割等，能够帮助项目优化性能，减小打包体积。

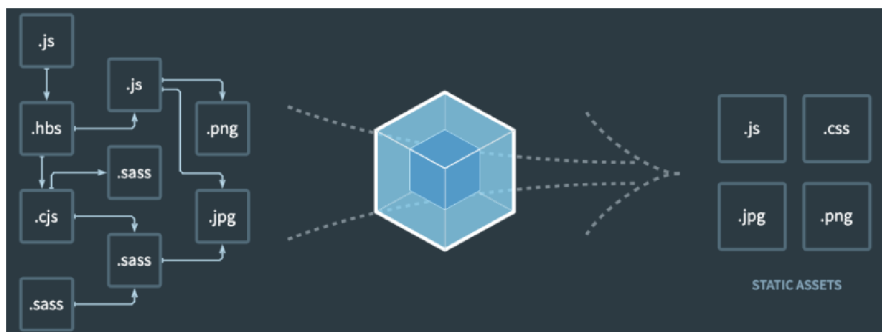
2. 依赖图

oderwhy

Webpack的依赖图

Webpack到底是如何对我们的项目进行打包的呢？

- 事实上webpack在处理应用程序时，它会根据命令或者配置文件找到入口文件；
- 从入口开始，会生成一个 **依赖关系图**，这个**依赖关系图**会包含应用程序中所需的所有模块（比如.js文件、css文件、图片、字体等）；
- 然后遍历图结构，打包一个个模块（根据文件的不同使用不同的loader来解析）；



3. 配置方式

oderwhy

loader配置方式

■ 配置方式表示的意思是在我们的webpack.config.js文件中写明配置信息：

- module.rules中允许我们配置多个loader（因为我们会继续使用其他的loader，来完成其他文件的加载）；
- 这种方式可以更好的表示loader的配置，也方便后期的维护，同时也让你对各个Loader有一个全局的概览；

■ module.rules的配置如下：

■ rules属性对应的值是一个数组：[Rule]

■ 数组中存放的是一个一个的Rule，Rule是一个对象，对象中可以设置多个属性：

- test属性：用于对 resource（资源）进行匹配的，通常会设置成正则表达式；
- use属性：对应的值是一个数组：[UseEntry]
 - ✓ UseEntry是一个对象，可以通过对象的属性来设置一些其他属性
 - loader：必须有一个 loader属性，对应的值是一个字符串；
 - options：可选的属性，值是一个字符串或者对象，值会被传入到loader中；
 - query：目前已经使用options来替代；
 - ✓ 传递字符串（如：use: ['style-loader']）是 loader 属性的简写方式（如：use: [{ loader: 'style-loader' }]）；
- loader属性：Rule.use: [{ loader }] 的简写。

I.配置样式loader



认识style-loader

- 我们已经可以通过css-loader来加载css文件了
 - 但是你会发现这个css在我们的代码中并没有生效（页面没有效果）。
- 这是为什么呢？
 - 因为css-loader只是负责将.css文件进行解析，并不会将解析之后的css插入到页面中；
 - 如果我们希望再完成插入style的操作，那么我们还需要另外一个loader，就是style-loader；



配置style-loader

- 那么我们应该如何使用style-loader：
 - 在配置文件中，添加style-loader；
 - 注意：因为loader的执行顺序是从右向左（或者说从下到上，或者说从后到前的），所以我们需要将style-loader写到css-loader的前面；

```
use: [  
  // 注意：style-loader在css-loader的前面  
  { loader: "style-loader" },  
  { loader: "css-loader" }  
]
```

补充：浏览器前缀（Vendor Prefix）是在 CSS 样式属性名称前面添加特定的前缀，以适应不同浏览器对新特性的支持情况。由于不同的浏览器对 CSS3 新特性的支持存在差异，为了确保网页在各种浏览器中都能正确显示，开发者需要添加相应的浏览器前缀。

例如，对于 CSS3 的 transition 属性，不同浏览器的写法如下：

- 标准写法：transition: width 2s;
- Webkit 内核浏览器（如 Chrome 和 Safari）写法：-webkit-transition: width 2s;
- Mozilla Firefox 写法：-moz-transition: width 2s;
- Opera 写法：-o-transition: width 2s;

- Internet Explorer 写法: `-ms-transition: width 2s;`

通过添加这些浏览器前缀, 确保了在使用 CSS3 新特性时, 在各种浏览器中都能获得相似的效果和体验。然而, 随着浏览器对 CSS3 新特性支持的不断完善, 一些较新版本的浏览器已经不再需要添加前缀即可支持这些属性, 因此在实际开发中, 需要根据具体情况选择是否添加浏览器前缀。

以下是一些 PostCSS 插件的功能:

1. **autoprefixer**: 自动添加浏览器前缀, 使得 CSS 样式在不同浏览器下表现一致。
2. **cssnano**: 压缩 CSS 文件, 减小文件体积, 提高加载速度。
3. **postcss-preset-env**: 允许使用未来的 CSS 语法, 如变量、嵌套规则、自定义选择器等, 而无需等待浏览器支持。
4. **postcss-px-to-viewport**: 将 px 单位转换为 vw 单位, 实现移动端适配。
5. **postcss-import**: 支持在 CSS 中使用 @import 语法引入其他 CSS 文件。

II.配置图片loader

使用asset而不是loader

```
{  
    test: /\.?(png|jpe?g|svg|gif)$/,  
    // 1.打包两张图片, 并且这两张图片有自己的地址, 将地址设置到  
img/bg1中  
    // 缺点: 多图片加载的两次网络请求  
    // type: "asset/resource",  
  
    // 2.将图片进行base64的编码, 并且直接编码后的源码放到打包的  
js文件中
```

// 缺点：造成js文件非常大，下载js文件本身消耗时间非常长，造成js代码的下载和解析/执行时间过长

```
// type: "asset/inline"
```

// 3.合理的规范：

// 3.1.对于小一点的图片，可以进行base64编码

// 3.2.对于大一点的图片，单独的图片打包，形成url地址，单独的请求这个url图片

```
type: "asset",
parser: {
  dataUrlCondition: {
    maxSize: 60 * 1024
  }
},
generator: {
  // 占位符
  // name: 指向原来的图片名称
  // ext: 扩展名
  // hash: webpack生成的hash
  filename: "img/[name]_[hash:8][ext]"
},
```

- **parser** 部分定义了数据 URL 的条件，当图片资源的大小小于等于 60KB 时，会将其转换为 base64 编码内联到打包的 JS 文件中。
- **generator** 部分定义了生成的图片文件的名称规则，其中使用了占位符 **[name]** 指向原始图片的名称，**[hash:8]** 表示 webpack 生成的 hash 值的前 8 位，保证文件名的唯一性。

这种配置方式综合了对于小图片使用 base64 编码内联到 JS 文件中，对于大图片仍然单独打包成 URL 地址的优化策略。这样可以在一定程度上减少 HTTP 请求次数，同时避免 JS 文件过大导致加载性能下降的问题。

III.配置babel loader

coderwhy

babel-loader

■ 在实际开发中，我们通常会在构建工具中通过配置babel来对其进行使用的，比如在webpack中。

■ 那么我们就需要去安装相关的依赖：

□ 如果之前已经安装了@babel/core，那么这里不需要再次安装；

```
npm install babel-loader -D
```

■ 我们可以设置一个规则，在加载js文件时，使用我们的babel：

```
module: {
  rules: [
    {
      test: /\.m?js$/,
      use: {
        loader: "babel-loader"
      }
    }
  ]
},
```

IV. 模块解析

负责我们在项目中使用require或者import时webpack怎么解析

coderwhy

resolve模块解析

■ resolve用于设置模块如何被解析：

□ 在开发中我们会有各种各样的模块依赖，这些模块可能来自于自己编写的代码，也可能来自第三方库；

□ resolve可以帮助webpack从每个 require/import 语句中，找到需要引入到合适的模块代码；

□ webpack 使用 [enhanced-resolve](#) 来解析文件路径；

■ webpack能解析三种文件路径：

■ 绝对路径

□ 由于已经获得文件的绝对路径，因此不需要再做进一步解析。

■ 相对路径

□ 在这种情况下，使用 import 或 require 的资源文件所处的目录，被认为是上下文目录；

□ 在 import/require 中给定的相对路径，会拼接此上下文路径，来生成模块的绝对路径；

■ 模块路径

□ 在 resolve.modules中指定的所有目录检索模块；

✓ 默认值是 ['node_modules']，所以默认会从node_modules中查找文件；

□ 我们可以通过设置别名的方式来替换初始模块路径，具体后面讲解alias的配置；



确实文件还是文件夹

- 如果是一个文件：
 - 如果文件具有扩展名，则直接打包文件；
 - 否则，将使用 `resolve.extensions` 选项作为文件扩展名解析；
- 如果是一个文件夹：
 - 会在文件夹中根据 `resolve.mainFiles` 配置选项中指定的文件顺序查找；
 - ✓ `resolve.mainFiles` 的默认值是 `['index']`；
 - ✓ 再根据 `resolve.extensions` 来解析扩展名；

`extensions`：这是一个数组，包含了在引入模块时可以省略的文件扩展名。这意味着当你在引入一个模块时，可以不写扩展名，webpack 会按照配置的顺序依次尝试解析这些文件，直到找到匹配的文件为止。

V.配置插件Plugins

- 上面表达的含义翻译过来就是：
 - Loader 是用于 **特定的模块类型** 进行转换；
 - Plugin 可以用于 **执行更加广泛的任务**，比如打包优化、资源管理、环境变量注入等；

HtmlWebpackPlugin 插件会使用默认的模板生成最终的 HTML 文件。默认情况下，HtmlWebpackPlugin 会生成一个简单的 HTML 文件，其中只包含一个 `script` 标签引用打包后的 JS 文件。

VI.搭建本地服务器

devServer 是 webpack 提供的一个开发服务器，用于在开发过程中快速启动一个本地服务器并实时监测文件变化，从而提供实时预览和热重载功能。

```
devServer: {  
  hot: true,  
  // host: "0.0.0.0",  
  // port: 8888,  
  // open: true  
  // compress: true  
},
```

这是什么意思

- `hot: true`: 这个配置表示启用模块热替换（Hot Module Replacement, HMR）功能。当某个模块发生变化时，只会重新加载该模块，而不会刷新整个页面，从而提高开发效率。
- `// host: "0.0.0.0"`: 这行代码被注释掉了，通常用于指定 devServer 的主机地址。如果需要通过局域网或者其他设备访问你的开发服务器，可以将 host 设置为 "0.0.0.0"，允许来自任意网络接口的访问。
- `// port: 8888`: 这行代码也是被注释掉的，用于指定 devServer 启动的端口号。默认情况下，devServer 会使用 8080 端口，如果需要指定其他端口，可以取消注释并设置相应的端口号。
- `// open: true`: 这行代码同样被注释掉，如果设置为 true，devServer 在启动后会自动打开默认浏览器并访问指定的 URL。
- `// compress: true`: 这行代码也是被注释掉的，如果设置为 true，devServer 会启用 gzip 压缩，以减小资源传输大小，加快网页加载速度。

4.深入webpack原理

[点击这里看webpack原理](#)

五.版本控制---git

1.什么是git

■ 不同版本的存储管理：

- 一个项目会不断进行版本的迭代，来修复之前的一些问题、增加新的功能、需求，甚至包括项目的重构；
- 如果我们通过手动来维护一系列的项目备份，简直是一场噩梦；

■ 重大版本的备份维护：

- 对于很多重大的版本，我们会进行备份管理；

■ 恢复之前的项目版本：

- 当我们开发过程中发生一些严重的问题时，想要恢复之前的操作或者回到之前某个版本；

■ 记录项目的点点滴滴：

- 如果我们每一个功能的修改、bug的修复、新的需求更改都需要记录下来，版本控制可以很好的解决；

■ 多人开发的代码合并：

- 项目中通常都是多人开发，将多人代码进行合并，并且在出现冲突时更好的进行处理；



集中式版本控制

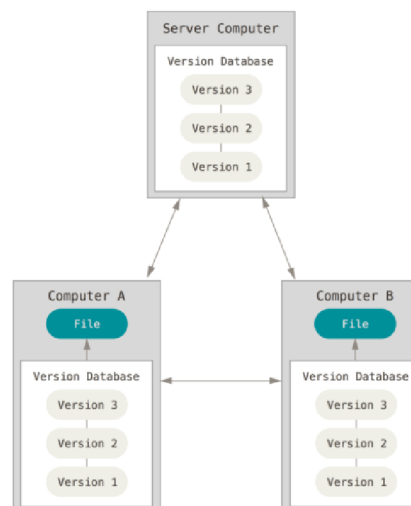
■ CVS和SVN都是属于集中式版本控制系统（Centralized Version Control Systems，简称 CVCS）

- 它们的主要特点是单一的**集中管理的服务器**，**保存所有文件的修订版本**；
- 协同开发人员通过客户端**连接到这台服务器**，**取出最新的文件或者提交更新**；

■ Git是属于分布式版本控制系统 (Distributed Version Control System, 简称 DVCS)

- 客户端并不只提取最新版本的文件快照，而是把代码仓库完整地镜像下来，包括完整的历史记录；
- 这么一来，任何一处协同工作用的服务器发生故障，事后都可以用任何一个镜像出来的本地仓库恢复；
- 因为每一次的克隆操作，实际上都是一次对代码仓库的完整备份；

■ 目前在公司开发中我们都是使用Git来管理项目的，所以接下来我们会重点学习Git的各种用法；



2.本次仓库

文件的状态划分

■ 现在我们的电脑上已经有一个Git仓库：

- 在实际开发中，你需要将某些文件交由这个Git仓库来管理；
- 并且我们之后会修改文件的内容，当达成某一个目标时，想要记录下来这次操作，就会将它提交到仓库中；

■ 那么我们需要对文件来划分不同的状态，以确定这个文件是否已经归于Git仓库的管理：

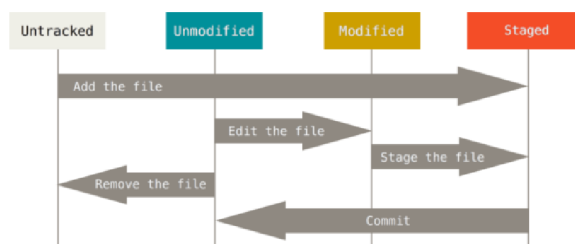
- **未跟踪**：默认情况下，Git仓库下的文件也没有添加到Git仓库管理中，我们需要通过add命令来操作；
- **已跟踪**：添加到Git仓库管理的文件处于已跟踪状态，Git可以对其进行各种跟踪管理；

■ 已跟踪的文件又可以进行细分状态划分：

- **staged**：暂缓区中的文件状态；
- **Unmodified**：commit命令，可以将staged中文件提交到Git仓库
- **Modified**：修改了某个文件后，会处于Modified状态；

■ 在工作时，你可以选择性地将这些修改过的文件放入暂存区；

■ 然后提交所有已暂存的修改，如此反复；



改变了文件之后必须先放入暂存区才可以提交

■ 如果我们修改文件的add操作，加上commit的操作有点繁琐，那么可以将两个命令结合来使用：

```
git commit -a -m "修改了bbb文件"
```

```
$ git commit -a -m "修改了bbb文件"
[master 24144aa] 修改了bbb文件
1 file changed, 1 insertion(+)
```



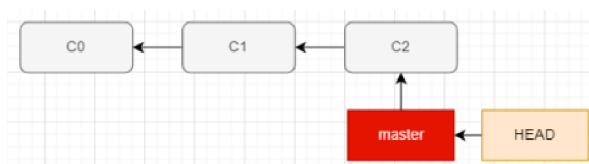
Git的校验和

- Git 中所有的数据在存储前都计算校验和，然后以 **校验和** 来引用。
 - Git 用以计算校验和的机制叫做 SHA-1 散列（hash，哈希）；
 - 这是一个由 40 个十六进制字符（0-9 和 a-f）组成的字符串，基于 Git 中文件的内容或目录结构计算出来；



版本回退 – git reset

- 如果想要进行版本回退，我们需要先知道目前处于哪一个版本：Git通过HEAD指针记录当前版本。
 - HEAD 是当前分支引用的指针，它总是指向该分支上的最后一次提交；
 - 理解 HEAD 的最简方式，就是将它看做 **该分支上的最后一次提交** 的快照；



- 我们可以通过HEAD来改变Git目前的版本指向：
 - 上一个版本就是HEAD^，上上一个版本就是HEAD^^；
 - 如果是上1000个版本，我们可以使用HEAD~1000；
 - 我们可以指定某一个commit id；

```
git reset --hard HEAD^
git reset --hard HEAD~1000
git reset --hard 2d44982
```

如果回退之后又需要再返回回退之前的版本，可以使用 `git reset --hard xxxx`，其中xxxx是版本对应的校验和

可以使用 `git log` 或者 `git reflog` 来查看记录

git log 和 git reflog的区别：

`git log` 和 `git reflog` 是 Git 中两个常用的命令，它们用于查看提交历史和引用日志，但它们的功能和用途有所不同：

(1) git log:

- `git log` 用于查看提交历史。它会按照提交的时间顺序列出每个提交的信息，包括提交者、提交日期、提交信息等。

(2) `git reflog`:

- `git reflog` 用于查看引用日志，即记录了 HEAD 和分支引用的变动历史。
- 引用日志会记录 HEAD 和分支的移动、重置、合并等操作，以及提交历史的哈希值。（HEAD指向当前分支的最后一次提交）
- `git reflog` 可以帮助开发者回退到之前的某个状态，即使已经丢失了对应的提交记录。

3. 远程仓库

当使用 `git clone` 命令克隆一个远程仓库时，Git 会自动将远程仓库添加为默认远程仓库，并且给这个远程仓库起一个默认的名称 `origin`



管理远程服务器

- 查看远程地址：比如我们之前从GitHub上clone下来的代码，它就是有自己的远程仓库的：

```
git remote
git remote -v
-v是-verbose的缩写(冗长的)
```

```
$ git remote
origin
```

```
$ git remote -v
origin https://github.com/coderwhy/hy-react-web-music.git (fetch)
origin https://github.com/coderwhy/hy-react-web-music.git (push)
```

- 添加远程地址：我们也可以继续添加远程服务器（让本地的仓库和远程服务器仓库建立连接）：

```
git remote add <shortname> <url>
git remote add gitlab http://152.136.185.210:7888/coderwhy/gitremotedemo.git
```

```
$ git remote -v
gitlab http://152.136.185.210:7888/coderwhy/gittestssh.git (fetch)
gitlab http://152.136.185.210:7888/coderwhy/gittestssh.git (push)
origin https://github.com/coderwhy/hy-react-web-music.git (fetch)
origin https://github.com/coderwhy/hy-react-web-music.git (push)
```

- 重命名远程地址：`git remote rename gitlab glab`

- 移除远程地址：`git remote remove gitlab`



远程仓库的验证 – 凭证

- 因为本身HTTP协议是无状态的连接，所以每一个连接都需要用户名和密码：
 - 如果每次都这样操作，那么会非常麻烦；
 - 幸运的是，Git 拥有一个凭证系统来处理这个事情；
- 下面有一些 Git Credential 的选项：



远程仓库的验证 – SSH密钥

- Secure Shell（安全外壳协议，简称SSH）是一种加密的网络传输协议，可在不安全的网络中为网络服务提供安全的传输环境。
- SSH以非对称加密实现身份验证。
 - 例如其中一种方法是使用自动生成的公钥-私钥对来简单地加密网络连接，随后使用密码认证进行登录；
 - 另一种方法是人工生成一对公钥和私钥，通过生成的密钥进行认证，这样就可以在不输入密码的情况下登录；
 - 公钥需要放在待访问的电脑之中，而对应的私钥需要由用户自行保管；
- 如果我们以SSH的方式访问Git仓库，那么就需要生产对应的公钥和私钥：

```
ssh-keygen -t ed25519 -C "your email"
```

```
ssh-keygen -t rsa -b 2048 -C "your email"
```

脑 > 本地磁盘 (C:) > 用户 > coderwhy > .ssh

名称	修改日期	类型
id_ed25519	2022/6/26 10:53	文件
id_ed25519.pub	2022/6/26 10:53	文件

用户设置 > SSH密钥

Q Search page

SSH密钥

SSH密钥用于在您的电脑和GitLab建立安全连接。

添加SSH密钥

添加 SSH 密钥，安全访问 GitLab。了解更多信息。

密钥

以 ssh-rsa, 'ssh-dss', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', 或 sk-ssh-ed25519@openssh.com 开头。

4.分支

什么是上游分支

上游分支（upstream branch）是指本地分支所关联的远程仓库中的分支。

当本地分支关联了一个上游分支时，在推送或者拉取代码时就不需要每次都指定远程仓库和分支，Git 已经知道应该从哪个远程仓库的哪个分支推送或者拉取。

```
git branch --set-upstream-to=<remote>/<branch>
<local_branch>
```

新建分支

```
git checkout -b <branch_name>
```

- `git checkout -b`: 这个选项告诉 Git 创建一个新的分支并切换到这个分支上。

删除分支

```
git branch -d
```

如果需要删除远程仓库上的分支:

```
git push origin --delete branch_name
```

切换分支

```
git checkout main
```

合并分支

1. **切换到要合并到的目标分支**: 首先, 确保你当前位于要合并到的目标分支上。假设要将 `feature` 分支合并到 `main` 分支, 则需要先切换到 `main` 分支:

```
git checkout main
```

2. **合并分支**: 使用 `git merge` 命令合并目标分支到当前分支 (即 `main` 分支):

```
git merge feature
```

合并分支除了merge 还有 **rebase**

使用 rebase 命令将提交到某一分支上的所有修改都移至另一分支上

- **git merge**: 这个命令会创建一个新的提交，这个提交叫merge，将两个分支的历史记录并列在一起，其中包含了合并前的各自分支的提交历史。这种方式**保留了原有的提交历史**，因此可以很清晰地看到分支的合并情况。因此再提交历史中往往能看到**分叉点**
- **git rebase**: 这个命令会将当前分支上的提交逐个应用到目标分支上，使得提交历史更加**线性**，看起来更加清晰。但是，**这也会改变提交历史**，因为它重新应用了提交，相当于创建了新的提交。

使用场景：

- **git merge**: 适用于合并公共分支（如 `master` 或 `develop`）或者在合并时不关心提交历史的情况。
- **git rebase**: 通常用于从一个分支中取出提交，并将它们应用到另一个分支上，使得提交历史保持线性，更加整洁。这在**合并特性分支**时很有用，可以**产生一个干净的提交历史**。

怎么解决合并冲突

解决合并冲突是一个常见的 Git 工作流程中的一部分。当你尝试合并两个分支时，如果 Git 发现它们之间存在冲突（即同一文件的同一部分在不同分支上有不同的修改），它会暂停合并过程并提示你手动解决这些冲突。

以下是解决 Git 合并冲突的一般步骤：

1. **编辑冲突文件**：打开冲突文件，并寻找冲突标记。
2. **标记为已解决**：在解决完所有冲突后，保存文件并将其标记为已解决。
对于每个文件，你可以使用 `git add` 命令将其标记为已解决：

```
git add <conflicted_file>
```


3. **完成合并**：一旦所有冲突都已解决并标记为已解决，可以继续完成合并操作。使用 `git merge --continue` 或者 `git commit` 命令来完成合并。如果使用 `git merge --continue`，Git 会自动提交合并结果，而如果使用 `git commit`，你可以添加合并的描述信息。
4. **推送合并后的更改（如果需要）**：如果你将更改推送到远程仓库，则可以使用 `git push` 命令。

5.Tag

在Git中，标签（tag）是用于标记特定提交（commit）的静态快照，通常用于标记版本发布。标签是 Git 版本库中的一个重要组成部分，可以帮助你轻松地找到特定版本的代码，并且不会随着代码的变动而移动。

以下是使用 Git 标签的一些常见操作和用法：

1. 创建标签：

- 使用 `git tag` 命令创建轻量标签：`git tag <tag_name>`
- 创建带注释的附注标签：`git tag -a <tag_name> -m "<message>"`

2. 查看标签：

- 查看所有标签：`git tag`
- 查看特定标签的详细信息：`git show <tag_name>`

3. 推送标签到远程仓库：

- 单个标签推送：`git push origin <tag_name>`
- 推送所有标签：`git push origin --tags`

4. 删除标签：

- 删除本地标签：`git tag -d <tag_name>`
- 删除远程仓库中的标签：`git push origin --delete <tag_name>`

5. 检出特定标签：

- 检出标签创建一个名为标签名的新分支：`git checkout -b <branch_name> <tag_name>`

- 检出标签在当前分支：`git checkout <tag_name>`

6. 列出与特定模式匹配的标签：

- 使用通配符（例如，`*`）来列出匹配的标签：`git tag -l "v1.*"`

7. 附注标签 vs 轻量标签：

- 附注标签（annotated tags）：可以添加消息、签名等附加信息，适用于重要的发布版本。
- 轻量标签（lightweight tags）：只是一个指向特定提交的引用，没有附加信息，适用于临时或内部使用。

8. 标签的最佳实践：

- 使用语义化版本号（Semantic Versioning）来命名标签，例如：`v1.0.0`。
- 标签应该与发布的版本一致，不应该在开发过程中频繁地创建标签。
- 确保在每次发布后都创建一个新的标签，以便轻松地跟踪版本历史。

使用标签可以帮助你更好地管理代码版本，方便地查找和回溯历史版本，以及确保团队成员可以轻松找到重要的发布版本。

6. 原理

(1). Git使用**提交对象**来构建代码库的版本历史记录，开发者可以通过提交对象来查看代码库的演化历程，进行版本回滚、分支合并等操作。提交对象是Git中的不可变对象，一旦创建就不能被修改。每个提交对象都有一个唯一的SHA-1哈希值，通过这个哈希值可以唯一地标识一个提交对象。

提交对象通常包含以下几个重要的信息：

- **快照（Snapshot）**：提交对象包含了当前代码库在**提交时的文件状态快照**，即存储了所有文件的内容。
- **指针**：除了快照之外，提交对象还包含一个或多个**指向其父提交对象的指针**。在大多数情况下，一个提交对象只有一个父提交对象，这意味着它是基于之前的一个提交而创建的。但在合并（merge）提交中，一个提交对象可能有多个父提交对象，代表了多个分支的合并。

- **作者和提交者信息 (Author and committer information)**：提交对象包含了提交者 (committer) 和作者 (author) 的信息，以及提交信息，也就是commit -m 后面的内容

(2). 除了提交对象以外，还有HEAD指针，**HEAD指针**默认指向当前分支最新的提交对象。